



HDU03-题解

- 预期难度：
 easy: 1003, 1005
 medium easy: 1006, 1009
 medium: 1004, 1010, 1002, 1008
 medium hard: 1011, 1012
 hard: 1001, 1007
- 由于出题人们语文水平有限，请搭配 `std` 以及大模型工具进行阅读。

1003

注意每次新加一个元素后，`b` 序列和至少翻倍，因此 `b` 序列的长度是 `log` 数量级的。

因此可以令 $f_{i,j}$ 表示考虑到第 i 个数字时，当前 `b` 序列长度为 j 的情况下，`b` 序列内数字和为多少，直接转移即可。

时间复杂度 $O(n \log W)$ 。

1005

将数字 n 分解质因数，有 $n = p_1^{k_1} \times p_2^{k_2} \times \dots$

发现每次操作，都会让每个质因子的次数至少除以 2。因此最优的分解策略是 $a = p_1^{q_1} \times p_2^{q_2} \times \dots$ ，其中 $q_i = \lfloor \frac{k_i}{2} \rfloor$ 。题目此时变为找到 n 中最大的质因子次数，给出它要经过多少次除以 2 下取整后变为 0。

当然可以通过 `pollar-rho` 来分解质因数，下面提供一个初学者友好的方法：

考虑只枚举 $p \leq \sqrt[3]{n}$ ，然后把那些质因子全部从 n 中去除。

剩下的数字只有可能是某一个质数的平方，或者是一个大质数。接下来只需要验证剩下的这个数字是否为某个质数的平方即可。

时间复杂度 $O(T \sqrt[3]{n})$ 。

1006

将 $[1, n]$ 中所有的整数都划分为 $x \times 2^k$ 形式，其中 x 为奇数。把 x 相同的整数看作一组，那么一共有 $\lceil \frac{n}{2} \rceil$ 组。

首先，同一组内任意两个不同的数字一定有整除关系，所以每一组最多能取出一个，答案不超过 $\lceil \frac{n}{2} \rceil$ 。

其次， $[\lceil \frac{n}{2} \rceil, n]$ 中的所有整数显然是一个合法解，说明最大集合的大小就是 $\lceil \frac{n}{2} \rceil$ 。

考虑字典序最小的情况：提取出来所有奇数 x ，假设最终集合内拥有数字 $x \times 2^{f_x}$ 。

为了保证集合内数字两两互不整除，一定有 $f_x > f_{kx}$ 。

根据上述条件，可以通过从大到小递推的方式得到所有 f_x 的最小值，也可以**观察到** $f_x = \lfloor \log_3(\frac{n}{x}) \rfloor$ 。

把所有的 $x \times 2^{f_x}$ 放一起排序后输出即可。

1009

“找到一种策略使得通过题目数量的期望最大”。

假设某道题可能的选项是 (a, b) ，那么将 a, b 两节点连边。

由于题和选项一一对应，用以上方式连出来的所有连通块均为基环树。原题变成给每条边定向，要求每个节点的度数都为 1。

此时可以发现，对于基环树的树边，定向方向只能是外向的（由于每个点的度数都必须要是 1）；环边则需要全部为顺时针或者逆时针。

因此，对于树上的节点对答案的贡献为 1，环上的节点对答案的贡献为 $\frac{1}{2}$ 。

难道树边对应的题目选项固定，环上的瞎蒙，不算是一种策略吗。。。

1004

费用流模板题。

考虑类似二分图的建法，假设 (e, w) 表示一条容量为 e ，费用为 w 的边。

首先源点 S 向每个小朋友连接 $(1, 0)$ ，表示每个小朋友都要买玩具；

其次每个小朋友向每个喜欢的玩具连接 $(1, 0)$ ，表示可以买这个玩具；

最后是每个玩具向汇点 T 连边，第 i 个玩具要连 y_i 条边，其中第 j 条边为 $(1, w_{i,j})$ 。

由于 w_i 数组单调递增，这样连边没有问题。

1010

首先需要知道 [这个](#) 结论，假设 c 为小 M 哼歌长度期望，放到本题来说就是如果小 H 喜欢的旋律是固定的话（假设这段旋律是 s ），有 $c = \sum_{s_{1..i}=s_{l-i+1..l}} w^i$ 。

考虑有多少长度为 l 的字符串拥有一个长度为 k 的 `border`：

- 字符串拥有长度为 k 的 `border` 等价于拥有长度为 $l - k$ 的周期，故这种字符串的数量为 w^{l-k} 。

注意到上述结论对于 $l = k$ 时不适用，因为所有字符串都有长度为 l 的 `border`。

因此，有：

$$\text{sum}(c) = \sum_{i=1}^{l-1} w^i \times w_{l-i} + w^{2l} = w^l(w^l + l - 1)。$$

由于小 H 喜欢的旋律随机，一共有 w^l 种不同的旋律，因此小 M 哼歌长度期望 $E(c) = \frac{\text{sum}(c)}{w^l} = w^l + l - 1$ 。

1002

题意分析

设：

- a_i 表示押第 i 支球队夺冠的钱；
- b_i 表示押第 i 支球队不夺冠的钱。

如果最终第 k 支球队夺冠，那么总回报为

$$x_k a_k + \sum_{i \neq k} \frac{x_i}{x_i - 1} b_i.$$

要求在

$$\sum_{i=1}^n (a_i + b_i) = w$$

的条件下，最大化所有可能冠军情况下的最小回报。

线性缩放

下注金额和最终回报都具有线性关系。

设保证最终回报至少为 1 所需要的最小投入为 C ，那么投入 w 时，能够保证的最大回报为

$$\left\lceil \frac{w}{C} \right\rceil.$$

理由如下：

- 将保证回报 1 的最优方案放大 $\frac{w}{C}$ 倍，就可以投入 w 并保证回报 $\frac{w}{C}$ ；
- 如果投入 w 可以保证回报 R ，将所有下注缩小为原来的 $\frac{1}{R}$ ，就可以投入 $\frac{w}{R}$ 并保证回报 1，所以 $C \leq \frac{w}{R}$ ，即 $R \leq \frac{w}{C}$ 。

所以问题变成求 C 。

建立线性规划

保证回报至少为 1 的最小投入满足

$$\begin{aligned}
& \min \quad \sum_{i=1}^n (a_i + b_i) \\
& \text{s.t.} \quad x_k a_k + \sum_{i \neq k} \frac{x_i}{x_i - 1} b_i \geq 1, \quad 1 \leq k \leq n, \\
& \quad \quad a_i, b_i \geq 0.
\end{aligned}$$

令

$$p_i = \frac{1}{x_i}.$$

注意到

$$\frac{x_i}{x_i - 1} = \frac{1}{1 - p_i}.$$

这个线性规划的对偶为

$$\begin{aligned}
& \max \quad \sum_{i=1}^n y_i \\
& \text{s.t.} \quad x_i y_i \leq 1, \\
& \quad \quad \frac{x_i}{x_i - 1} \sum_{j \neq i} y_j \leq 1, \\
& \quad \quad y_i \geq 0.
\end{aligned}$$

根据线性规划强对偶，对偶问题的最大值也是 C 。

设

$$S = \sum_{i=1}^n y_i.$$

第一个约束等价于

$$y_i \leq p_i.$$

第二个约束等价于

$$S - y_i \leq 1 - p_i,$$

即

$$y_i \geq S + p_i - 1.$$

结合 $y_i \geq 0$, 有

$$\boxed{\max(0, S + p_i - 1) \leq y_i \leq p_i}.$$

所以 S 可行, 当且仅当

$$\sum_{i=1}^n \max(0, S + p_i - 1) \leq S$$

并且

$$S \leq \sum_{i=1}^n p_i.$$

同时, 由 $y_i \leq p_i$ 和 $S - y_i \leq 1 - p_i$, 还可以得到

$$S \leq 1.$$

情况一: $\sum p_i \leq 1$

设

$$P = \sum_{i=1}^n p_i.$$

因为所有 $y_i \leq p_i$, 所以一定有

$$S \leq P.$$

下面证明 $S = P$ 确实可行。

令

$$y_i = p_i.$$

那么

$$\sum_{i=1}^n y_i = P.$$

第一个约束显然满足。

对于第二个约束，有

$$S - y_i = P - p_i \leq 1 - p_i,$$

其中使用了 $P \leq 1$ 。

所以 $S = P$ 可行，进而

$$C = P = \sum_{i=1}^n \frac{1}{x_i}.$$

最终答案为

$$\boxed{\frac{w}{\sum_{i=1}^n \frac{1}{x_i}}}.$$

这对应代码中的

```
if(s <= 1) {  
    return (db)w / s;  
}
```

情况二： $\sum p_i > 1$

现在考虑

$$\sum_{i=1}^n p_i > 1.$$

定义

$$q_i = 1 - p_i = 1 - \frac{1}{x_i} = \frac{x_i - 1}{x_i}.$$

那么可行条件中的下界可以写成

$$\max(0, S + p_i - 1) = \max(0, S - q_i).$$

所以 S 可行, 当且仅当

$$\sum_{i=1}^n \max(0, S - q_i) \leq S.$$

将 q_i 从小到大排序:

$$q_1 \leq q_2 \leq \cdots \leq q_n.$$

因为

$$q_i = \frac{x_i - 1}{x_i},$$

所以

$$\frac{1}{q_i} = \frac{x_i}{x_i - 1}.$$

按照 $\frac{x_i}{x_i - 1}$ 从大到小排序, 就等价于按照 q_i 从小到大排序。这正是代码中

```
for(int i: p) v.push_back(1.00 * i / (i-1));
sort(v.begin(), v.end(), greater<>());
```

所做的事情。

此时有

$$\frac{1}{v_i} = q_i.$$

将可行条件转化为前缀限制

对于任意一个集合 A , 有

$$\sum_{i \in A} (S - q_i) \leq \sum_{i=1}^n \max(0, S - q_i).$$

因此, 如果 S 可行, 那么对任意集合 A 都有

$$\sum_{i \in A} (S - q_i) \leq S.$$

设集合大小为 m , 则

$$mS - \sum_{i \in A} q_i \leq S.$$

当 $m \geq 2$ 时, 有

$$(m-1)S \leq \sum_{i \in A} q_i,$$

也就是

$$S \leq \frac{\sum_{i \in A} q_i}{m-1}.$$

对于固定的 m , 为了得到最严格的限制, 应选择 q_i 最小的 m 个数, 即排序后的前缀。

记

$$Q_m = \sum_{i=1}^m q_i.$$

那么必须满足

$$S \leq \frac{Q_m}{m-1}, \quad 2 \leq m \leq n.$$

因此

$$S \leq \min_{2 \leq m \leq n} \frac{Q_m}{m-1}.$$

下面证明这些条件也是充分的。

假设对所有 $m \geq 2$ 都满足

$$S \leq \frac{Q_m}{m-1}.$$

设恰好有前 m 个 q_i 小于 S , 那么

$$\sum_{i=1}^n \max(0, S - q_i) = \sum_{i=1}^m (S - q_i) = mS - Q_m.$$

由

$$S \leq \frac{Q_m}{m-1}$$

可得

$$(m-1)S \leq Q_m.$$

所以

$$mS - Q_m \leq S.$$

也就是

$$\sum_{i=1}^n \max(0, S - q_i) \leq S.$$

因此 S 可行。

综上, 最大的可行值为

$$C = \min_{2 \leq m \leq n} \frac{Q_m}{m-1}.$$

最终答案为

$$\frac{w}{C} = \frac{w}{\min_{2 \leq m \leq n} \frac{Q_m}{m-1}}.$$

由于所有数都为正数，取倒数后，最小值变成最大值：

$$\frac{w}{C} = \max_{2 \leq m \leq n} \frac{w(m-1)}{Q_m}.$$

1008

先从不带修的版本入手。

可以发现只需要考虑 x_1, x_n 两个变量，从高位到低位考虑，对于第 i 位，一共有三种情况：

- $x_{1,i} = 1, x_{n,i} = 1$
- $x_{1,i} = 1, x_{n,i} = 0$
- $x_{1,i} = 0, x_{n,i} = 0$

对于第一种和第三种情况，只能有所有 x 的第 i 位都同时为 1/0；对于第二种情况，可以预处理出来有多少种合法的取值（跑 dag 上的计数，每个变量都只能有 0/1 两种取值且后继不大于前驱的方案数），后文假设这个方案数为 c 。

令 $f_{i,0/1,0/1}$ 表示从高到低考虑到第 i 位；此时 x_1 是否小于 r ；此时 x_n 是否大于 l 的方案数，初始状态是 $f_{0,0,0} = 1$ 。

考虑以下转移：

```

for(int i = 0; i < len; i++){
    int tp = l[i] - '0', sp = r[i] - '0';
    for(int j = 0; j <= 1; j++){//枚举 f 的第二维
        for(int k = 0; k <= 1; k++){//枚举 f 第三维
            for(int o = 0; o <= 1; o++){//x_1 第 i 位
                for(int p = 0; p <= o; p++){//x_n 第 i 位
                    int fx = (o && !p)? c: 1;// 系数
                    if(!j && o > tp) continue;
                    // 如果此时仍有 x_1 = r, 那么 r_i = 0, x_{1, i} = 1 不合法
                    if(!k && p < sp) continue;// 和上述类似
                    int tj = j | (o < tp), tk = k | (p > sp);
                    //更新 x_1 和 r 的关系, x_n 和 l 的关系
                    add(f[i+1][tj][tk], f[i][j][k] * fx);
                }
            }
        }
    }
}

```

不难发现，可以把二维数组 `f_i` 拍平，写成 1×4 向量的形式，那么所有的转移都可以用矩阵乘法实现，用线段树维护转移矩阵的乘法即可实现带修。

1011

思路

将下标 i 看成三维点

$$P_i = (A_i, B_i, C_i).$$

由于 A, B, C 均为 $[0, n)$ 的排列，每一维的坐标都互不相同。

设某个可行的 mex 三元组为 (x, y, z) 。

如果某一维的 mex 等于 n ，说明所有下标都被选中，因此三元组只能是

$$(n, n, n),$$

固定贡献 1。下面只考虑 $x, y, z < n$ 。

定义三个见证位置：

$$\text{pos}_A(x), \quad \text{pos}_B(y), \quad \text{pos}_C(z).$$

它们可能重合，因此形成一个大小不超过 3 的点集 W 。这些位置不能被选入 S 。为了不与“小于 mex 的数必须出现”矛盾，任意 $i \in W$ 都必须满足

$$A_i \geq x, \quad B_i \geq y, \quad C_i \geq z.$$

因此

$$(x, y, z) = \left(\min_{i \in W} A_i, \min_{i \in W} B_i, \min_{i \in W} C_i \right).$$

如果 W 中某个点没有取得任何一维的最小值，删除它不会改变三元组。故不同的 mex 三元组恰好对应满足以下条件的点集：

- 点集大小为 1, 2 或 3;
- 每个点至少取得一个维度的最小值。

反过来，对于任意满足条件的点集，取其逐维最小值 (x, y, z) ，并选入所有满足 $A_i < x$ 、 $B_i < y$ 或 $C_i < z$ 的位置，即可得到该 mex 三元组，所以该对应是双射。

一个见证点

任意单点都合法，贡献为

$$n.$$

两个见证点

定义

$$d_{ABC}(i) = \#\{j \mid A_j > A_i, B_j > B_i, C_j > C_i\}.$$

一个双点集合不合法，当且仅当其中一个点在三个维度上都严格大于另一个点。每个这样的点对恰好被较小的点统计一次。

因此合法双点集合数为

$$\binom{n}{2} - \sum_i d_{ABC}(i).$$

三个见证点

再定义

$$d_{AB}(i) = \#\{j \mid A_j > A_i, B_j > B_i\},$$

并类似定义 $d_{AC}(i), d_{BC}(i)$ 。

一个三点集合不合法，当且仅当某个点没有取得任何维度的最小值。此时三个维度的最小值由至多两个点取得，因此必有一个点同时取得至少两个维度的最小值。

点 i 同时取得 A, B 两维最小值时，另外两个点可从 $d_{AB}(i)$ 个点中选择，方案数为

$$\binom{d_{AB}(i)}{2}.$$

另外两组同理。

如果点 i 同时取得三个维度的最小值，该集合会在 AB, AC, BC 三类中各统计一次，但只应计算一次，因此需要加回两次。此类集合数为

$$\binom{d_{ABC}(i)}{2}.$$

不同点不可能同时取得包含同一维度的最小值，而三类事件两两都有公共维度，所以除此之外没有其他交集。

故合法三点集合数为

$$\binom{n}{3} - \sum_i \left(\binom{d_{AB}(i)}{2} + \binom{d_{AC}(i)}{2} + \binom{d_{BC}(i)}{2} \right) + 2 \sum_i \binom{d_{ABC}(i)}{2}.$$

最终答案

综上，

$$\begin{aligned} \text{Ans} = & 1 + n + \binom{n}{2} - \sum_i d_{ABC}(i) + \binom{n}{3} \\ & - \sum_i \left(\binom{d_{AB}(i)}{2} + \binom{d_{AC}(i)}{2} + \binom{d_{BC}(i)}{2} \right) + 2 \sum_i \binom{d_{ABC}(i)}{2}. \end{aligned}$$

支配计数

对于 d_{AB} , 将点按 A 从大到小排序, 用树状数组维护已加入点的 B 坐标, 即可在 $O(n \log n)$ 内求出。 d_{AC}, d_{BC} 同理。

对于 d_{ABC} , 先按 A 从大到小排序, 再使用 CDQ 分治:

- 左半部分点的 A 一定大于右半部分;
- 归并时按 B 从大到小扫描;
- 用树状数组统计左半部分中 C 也更大的点。

其时间复杂度为 $O(n \log^2 n)$ 。

正确性说明

由见证点的定义, 每个可行三元组都唯一一对应其三个 mex 值所在位置组成的集合。删除未取得任何维度最小值的冗余点后, 得到大小至多为 3 且每个点至少取得一维最小值的点集。

反过来, 任意这样的点集都能按逐维最小值构造出对应的集合 S , 且由于每维坐标互不相同, 三个最小值的位置恰好恢复原点集。因此二者一一对应。

上述公式分别无重无漏地统计了大小为 1, 2, 3 的合法见证点集, 并额外加入 (n, n, n) , 故答案正确。

复杂度

每组数据的时间复杂度为

$$O(n \log^2 n),$$

空间复杂度为

$$O(n).$$

需要使用 `long long`。

1012

一、只需要保留凸包

对于任意一个方向，点集在这个方向上的最大投影和最小投影一定可以在凸包顶点上取得。

所以，凸包内部的点不会影响任何方向上的宽度，也就不会影响最终答案。

先对原点集求凸包，记为 D 。

二、用闵可夫斯基和表示点集的宽度

将 D 中每个点取相反数，得到

$$-D = \{-p \mid p \in D\}.$$

再计算闵可夫斯基和

$$K = D + (-D).$$

K 中的每个点都可以写成

$$p - q,$$

其中 $p, q \in D$ 。

现在固定一个方向。原点集在这个方向上的宽度，就是该方向上的最大投影减去最小投影，也就是

$$\max_{p, q \in D} (p - q) \text{ 在该方向上的投影}.$$

而所有 $p - q$ 恰好组成 $D + (-D)$ ，所以：

D 在任意方向上的宽度，等于 K 在该方向上的最大投影。

这样，我们就把“最大投影减最小投影”转化成了对一个凸包求“最大投影”。

三、同时处理横向跨度和纵向跨度

假设碎片旋转到某个角度后，当前水平方向在原坐标系中对应方向 u 。

那么：

- 当前横向跨度等于 K 在方向 u 上的最大投影；
- 当前纵向跨度等于 K 在与 u 垂直的方向上的最大投影。

将 K 中每个点逆时针旋转 90° ，得到 K' 。点

$$(x, y)$$

旋转后变为

$$(-y, x).$$

于是， K' 在方向 u 上的最大投影，就等于 K 在垂直于 u 的方向上的最大投影。

因此当前最小正方形的边长为

$$L(u) = \max(K \text{ 在 } u \text{ 方向上的最大投影}, K' \text{ 在 } u \text{ 方向上的最大投影}).$$

四、将两个最大值合并成一个凸包

将 K 和 K' 的所有顶点放在一起，再求一次凸包：

$$T = \text{ConvexHull}(K \cup K').$$

对于任意方向 u ， T 在该方向上的最大投影，就是 K 和 K' 在该方向上的最大投影中的较大值。

因此：

碎片旋转到方向 u 时，最小正方形的边长，恰好等于凸包 T 在方向 u 上的最大投影。

到这里，原问题已经转化为：

求一个凸包在所有方向上的最大投影的平均值。

五、最大投影积分与凸包周长

设凸包 T 在方向 θ 上的最大投影为

$$h(\theta).$$

根据前面的构造, $h(\theta)$ 就是对应时刻最小正方形的边长, 因此观察框的周长为

$$Q(\theta) = 4h(\theta).$$

对于任意凸包, 都有如下结论:

$$\int_0^{2\pi} h(\theta) d\theta = \text{perimeter}(T).$$

可以通过“宽度”来理解。凸包在方向 θ 上的宽度, 等于正方向最大投影与反方向最大投影之和, 即

$$w(\theta) = h(\theta) + h(\theta + \pi).$$

另一方面, 凸包周长等于它在长度为 π 的所有方向上的宽度积分:

$$\text{perimeter}(T) = \int_0^\pi w(\theta) d\theta.$$

代入 $w(\theta)$, 得到

$$\begin{aligned} \text{perimeter}(T) &= \int_0^\pi (h(\theta) + h(\theta + \pi)) d\theta \\ &= \int_0^{2\pi} h(\theta) d\theta. \end{aligned}$$

所以一整周内观察框周长的平均值为

$$\begin{aligned} \frac{1}{2\pi} \int_0^{2\pi} Q(\theta) d\theta &= \frac{1}{2\pi} \int_0^{2\pi} 4h(\theta) d\theta \\ &= \frac{2}{\pi} \text{perimeter}(T). \end{aligned}$$

最终答案为

$$\text{perimeter}(T) \times \frac{2}{\pi}.$$

六、复杂度分析

设原始点数为 n 。

- 第一次求凸包: $O(n \log n)$;
- 构造相反点集: $O(n)$;
- 计算两个凸包的闵可夫斯基和: $O(n)$;
- 合并 K 与 K' 后再次求凸包: $O(n \log n)$;
- 计算周长: $O(n)$ 。

因此总时间复杂度为

$$O(n \log n),$$

空间复杂度为

$$O(n).$$

1001

考虑朴素的求解, 记 $dp[i]$ 表示 $s[1, i]$ 的合法划分方案数, 转移为 $dp[i] = \sum_{0 \leq j < i, s[j+1, i] \text{ 不是重串}} dp[j]$ 。我们想办法优化枚举条件 " $s[j+1, i]$ 不是重串", 也就是想办法描述所有以 i 结尾的重串。

求出所有的 runs。根据相关理论, 每个重串属于且仅属于唯一一个 runs, 所有 runs 的 $length - period$ 之和是 $O(n \log n)$ 级别的。

于是在 i 处不可以转移的位置可以根据 runs 描述为一些等差数列。这些等差数列的长度和为 $O(n \log n)$ 。

我们希望对于每个 runs, 计算在 $s[1, i]$ 中等差数列当前的 dp 值前缀和。因为不存在下标偏移 (也就是我们需要的是绝对下标 $dp[j]$), 可以隐式地建出图, 在图上增量计算。最终用总的转移方案数减去不可以转移的方案数即可。

时间复杂度为 $O(n \log n)$ ，空间复杂度可以做到线性，如果显示建图空间为 $O(n \log n)$ ，理论上也可以通过。

可能存在时间复杂度相同或稍高的基于 SA 的做法。出题人水平有限想不到线性做法。

runs 相关理论参考 Hideo Bannai, Tomohiro I, Shunsuke Inenaga, Yuto Nakashima, Masayuki Takeda, Kazuya Tsuruta. "The 'Runs' Theorem." SIAM Journal on Computing, vol. 46, no. 5, 2017.

1007

思路

设水位为 H ，则位置 i 的石头露出当且仅当

$$h_i \geq H。$$

将石头按高度从大到小加入，并将高度相同的石头同时加入。处理完高度 H 后，当前状态恰好对应水位为 H 。

由于按水位从高到低枚举，因此一个询问第一次形成至少 K 个连续段时，当前的 H 就是答案。

连续段数

令 up_i 表示位置 i 是否露出，则区间 $[l, r]$ 中连续段数为

$$cnt(l, r) = up_l + \sum_{i=l+1}^r [up_i = 1 \wedge up_{i-1} = 0]。$$

用树状数组维护

$$b_i = [up_i = 1 \wedge up_{i-1} = 0]，$$

即可在 $O(\log n)$ 内计算

$$cnt(l, r) = up_l + \text{BIT}(r) - \text{BIT}(l)。$$

当位置 x 露出时，若 x 严格位于询问内部，段数变化如下：

左邻居	右邻居	变化
未露出	未露出	+1
已露出	未露出	0
未露出	已露出	0
已露出	已露出	-1

若 x 位于询问边界，则只考虑区间内的邻居。

维护询问

若区间 $B \subseteq A$ ，则任意时刻都有

$$cnt(B) \leq cnt(A)。$$

因此只需维护尚未回答询问中，在包含关系下极大的区间。被包含的询问等外层询问回答并删除后再加入，不会错过答案。

将询问按 l 升序、 r 降序排序。当前极大区间的左右端点均严格递增，所以包含某个位置的活跃询问在排序后构成连续区间，可以用线段树批量修改其连续段数。

维护以下数据结构：

- 一棵线段树：区间加，并查询活跃询问的最大连续段数；
- 一棵辅助线段树：删除询问后寻找新出现的极大区间；
- 两个有序集合：按左右端点定位需要修改的询问范围；
- 一个树状数组：计算新加入询问当前的连续段数。

每处理完一组同高度石头，若线段树最大值不小于 K ，就将对应询问的答案设为当前高度并删除，直到最大值小于 K 。

没有被回答的询问输出 -1 。

操作次数证明

每块石头只会露出一次。加入位置 x 时，只需根据 $x-1, x+1$ 的状态，将受影响的活跃询问划分为常数个连续区间，因此每块石头只产生 $O(1)$ 次线段树区间修改和树状数组修改。

每个询问最多经历以下过程：

1. **加入一次**：询问初始为极大区间，或在包含它的极大区间删除后成为新的极大区间；
2. **删除一次**：询问第一次满足条件后被永久删除；
3. **失效一次**：询问从线段树和活跃集合中删除，此后不会再次加入。

询问加入活跃集合后，在删除前会一直保持活跃，因此不会重复加入；删除只发生在其第一次满足条件时，因此也不会重复删除。

删除一个极大区间后，辅助线段树可能找出多个新的极大区间。虽然单次删除可能加入多个询问，但每次循环都会加入一个此前从未活跃过的询问。总共只有 m 个询问，因此所有恢复循环的执行次数之和不超过 m 。

所以：

- 所有石头共进行 $O(n)$ 次数据结构操作；
- 所有询问共进行 $O(m)$ 次加入、删除和恢复操作；
- 每次操作复杂度为 $O(\log n)$ 或 $O(\log m)$ 。

一个活跃询问可能受到很多石头影响，但这些影响通过线段树区间加批量完成，而不是逐个修改询问，因此不会产生 $O(nm)$ 的复杂度。

正确性

处理完高度 H 的所有石头后，有

$$up_i = 1 \iff h_i \geq H,$$

与水位为 H 时的露出状态完全相同。

线段树按照加入石头后连续段的实际变化维护每个活跃询问，因此其段数始终正确。

若 $B \subseteq A$ ，则 $cnt(B) \leq cnt(A)$ ，所以 B 满足条件时 A 必然也满足条件。因此，延迟加入被包含的询问不会遗漏更高的合法水位。

算法按水位从高到低处理，所以询问第一次满足

$$cnt(l, r) \geq K$$

时的水位就是最大合法水位。

复杂度

排序复杂度为 $O(n \log n + m \log m)$ 。

每块石头产生常数次数据结构操作，每个询问只会加入和删除一次，因此总时间复杂度为

$O(n \log n + m \log m)$ 。

空间复杂度为 $O(n + m)$ 。